

Jour 3 — Travaux pratiques

Version apprenants — énoncés d'exercices

Contenu

TP 04 · Blog avec React Router

TP 05 · Mini E-Commerce

TP 06 · Tester la Todo List

TP 07 · Migration du blog vers Next.js

Consigne générale : réalisez chaque exercice dans l'ordre. L'objectif est d'obtenir une application fonctionnelle, lisible et structurée. Le code doit rester propre, cohérent et exécutable à tout moment.

TP 04 — Blog avec React Router

Exercice 1 — Mettre en place la structure générale du blog

À partir d'une application React, mets en place une architecture multi-pages avec React Router.

L'application doit contenir au minimum :

- une page d'accueil
- une page blog
- une page de détail d'article
- une page about
- une page login
- une page profil
- une page 404

Le routage doit être configuré proprement dans le composant principal de l'application.

Exercice 2 — Créer un layout partagé

Crée un layout commun utilisé par plusieurs pages de l'application.

Ce layout doit contenir au minimum :

- un header
- une zone principale de contenu
- un footer

Les pages accueil, blog, détail d'article et about doivent s'afficher à l'intérieur de ce layout partagé.

Exercice 3 — Afficher la liste des articles

Crée une page /blog qui affiche une liste d'articles.

Chaque article doit présenter au minimum :

- un titre
- un extrait
- une catégorie
- un lien vers sa page de détail

Tu peux utiliser un tableau de données local pour simuler les articles.

Exercice 4 — Créer une page de détail dynamique

Mets en place une route dynamique permettant d'afficher un article précis à partir de son slug.

La page doit être accessible avec une URL de type : /blog/mon-article.

Sur cette page, affiche les informations correspondant à l'article sélectionné.

Exercice 5 — Ajouter une navigation active

Crée un menu de navigation avec des liens vers les principales pages du site.

Les liens actifs doivent être visuellement distingués des autres liens.

Exercice 6 — Filtrer les articles par catégorie avec les query params

Ajoute un système de filtrage sur la page blog.

L'utilisateur doit pouvoir filtrer les articles selon leur catégorie grâce aux paramètres d'URL.

Le filtre doit modifier l'URL et l'interface doit refléter correctement le filtre sélectionné.

Exercice 7 — Protéger une route avec une fausse authentification

Crée une route /profile accessible uniquement si l'utilisateur est considéré comme connecté.

Tu peux simuler l'authentification avec une variable, un state ou un faux contexte.

Si l'utilisateur n'est pas connecté, il doit être redirigé vers la page de login.

Exercice 8 — Gérer les routes inconnues

Ajoute une page 404 personnalisée.

Toute URL non prévue dans le routeur doit afficher cette page.

TP 05 — Mini E-Commerce

Exercice 1 — Afficher un catalogue de produits

Crée une mini application e-commerce affichant une liste de produits.

Chaque produit doit présenter au minimum :

- son nom
- son prix
- éventuellement son image
- sa catégorie
- un bouton d'ajout au panier

Les données peuvent être simulées localement.

Exercice 2 — Ajouter un système de filtre

Permetts à l'utilisateur de filtrer les produits affichés.

Tu peux prévoir par exemple :

- un filtre par catégorie
- un filtre par mot-clé
- ou les deux

L'affichage doit se mettre à jour dynamiquement selon les filtres choisis.

Exercice 3 — Mettre en place un panier

Ajoute un panier global dans l'application.

L'utilisateur doit pouvoir :

- ajouter un produit au panier
- voir les produits présents dans le panier
- supprimer un produit du panier

Le panier doit rester cohérent même si le même produit est ajouté plusieurs fois.

Exercice 4 — Gérer les quantités

Fais évoluer le panier pour permettre la gestion des quantités.

L'utilisateur doit pouvoir :

- augmenter la quantité d'un produit
- diminuer la quantité
- retirer complètement le produit si nécessaire

Exercice 5 — Calculer le total dynamiquement

Affiche le total du panier en fonction des produits présents et de leurs quantités.

Le total doit se recalculer automatiquement à chaque modification du panier.

Exercice 6 — Mettre en place un thème clair / sombre

Ajoute un bouton permettant de changer le thème de l'application.

Le thème sélectionné doit modifier l'apparence générale de l'interface.

Exercice 7 — Persister le panier

Fais en sorte que le contenu du panier soit conservé même après rechargement de la page.

Utilise le stockage local du navigateur pour sauvegarder et restaurer les données.

Exercice 8 — Centraliser le state global

Mets en place une gestion d'état global avec Zustand ou Redux Toolkit.

Le catalogue, le panier et éventuellement le thème doivent utiliser cette logique centralisée dès que cela est pertinent.

TP 06 — Tester la Todo List

Exercice 1 — Tester le reducer

Écris les tests unitaires du reducer de la Todo List.

Tous les cas importants doivent être couverts, notamment :

- ajout d'une tâche
- changement d'état d'une tâche
- suppression d'une tâche
- autres actions présentes dans ton reducer

Chaque test doit vérifier que le nouvel état est correct et que l'ancien état n'a pas été muté.

Exercice 2 — Tester le composant d'affichage d'une tâche

Écris des tests pour le composant qui représente une tâche individuelle.

Tes tests doivent vérifier au minimum :

- que le texte s'affiche correctement
- que l'état visuel correspond au statut de la tâche
- que l'action de toggle fonctionne
- que l'action de suppression fonctionne

Exercice 3 — Tester le formulaire d'ajout

Écris les tests du formulaire permettant d'ajouter une tâche.

Tu dois vérifier au minimum :

- qu'une saisie valide peut être soumise
- qu'une valeur vide est refusée
- que le champ est vidé après ajout
- que la fonction attendue est appelée correctement

Exercice 4 — Tester le parcours utilisateur complet

Crée des tests end-to-end avec Cypress pour simuler l'utilisation réelle de l'application.

Le scénario doit couvrir au minimum :

- l'ajout d'une tâche
- le changement d'état d'une tâche
- la suppression d'une tâche
- l'utilisation d'un filtre

TP 07 — Migration du blog vers Next.js

Exercice 1 — Initialiser le projet Next.js

Crée un nouveau projet Next.js avec la structure moderne vue en cours.

Le projet doit démarrer correctement en local.

Exercice 2 — Reprendre les composants du blog React

Récupère les composants utiles du blog React existant et intègre-les dans le nouveau projet Next.js.

Tu peux reprendre par exemple :

- le header
- les cartes d'articles
- la liste des articles
- les composants communs

Exercice 3 — Mettre en place le routing avec l'App Router

Recrée les routes principales du blog avec la structure de fichiers de Next.js.

L'application doit contenir au minimum :

- la page d'accueil
- la page about
- la page blog
- la page de détail d'un article

Le projet ne doit plus utiliser React Router.

Exercice 4 — Afficher la liste des articles côté serveur

Mets en place l'affichage de la liste des articles dans Next.js avec une récupération de données adaptée.

La page blog doit afficher correctement les articles au rendu.

Exercice 5 — Générer statiquement les pages d'articles

Configure les pages dynamiques pour générer les pages de détail à partir des slugs disponibles.

Chaque article doit être accessible via son URL dédiée.

Exercice 6 — Optimiser les images

Remplace les images classiques par le composant prévu par Next.js lorsque c'est pertinent.

L'affichage doit rester correct.

Exercice 7 — Ajouter une API Route

Crée une route d'API permettant d'exposer les articles.

Au minimum, une route GET /api/posts doit être disponible.

Exercice 8 — Gérer la page 404

Ajoute une page personnalisée pour les routes inexistantes.

Toute URL invalide doit afficher cette page.

Exercice 9 — Préparer le déploiement

Prépare le projet pour un déploiement sur Vercel.

Le projet doit être proprement structuré, build sans erreur et prêt à être poussé puis déployé.